

data communication between Angular components

1. Why Component Communication Is Needed

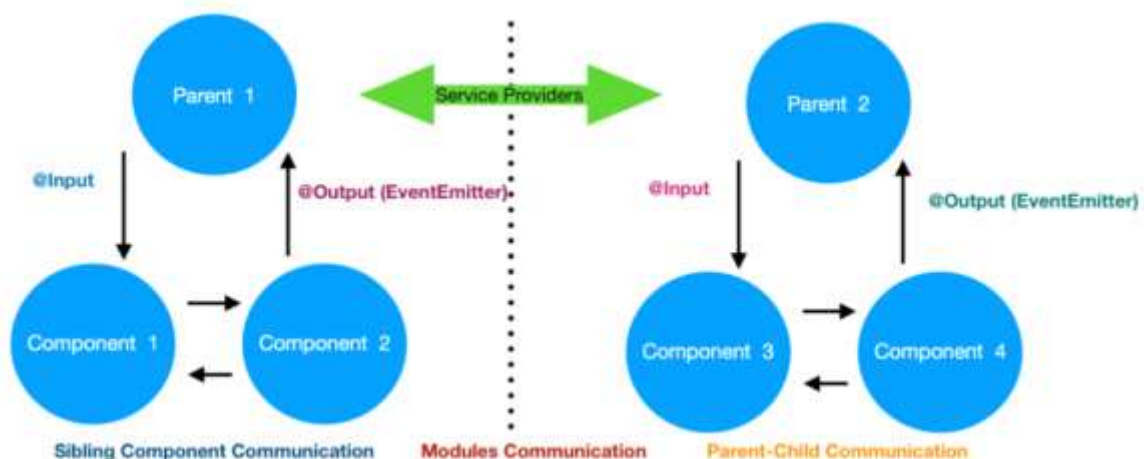
Angular applications are built as **component trees**. Each component:

- Has its own **template**
- Maintains **isolated state**
- Should follow **separation of concerns**

Because of this isolation, components **cannot directly access each other's data**. Angular therefore provides **well-defined communication mechanisms** depending on the **relationship between components**.

2. Types of Component Relationships

Component communication is chosen based on **hierarchy and scope**.



Relationship	Typical Use Case
Parent → Child	Pass data/configuration

Relationship	Typical Use Case
Child → Parent	Emit events/actions
Sibling → Sibling	Share state indirectly
Unrelated Components	Global/shared state
Route-based	Navigation & context passing

3. Parent → Child Communication (@Input)

Purpose

Used when the **parent owns the data** and passes it **downward**.

Mechanism

- Parent binds a value
- Child receives it via @Input()

Example

Parent Component

```
<app-user-profile [username]="loggedInUser"></app-user-profile>
```

Child Component

```
@Input() username!: string;
```

Key Characteristics

- One-way data flow
 - Child **must not modify** parent state
 - Most common and recommended approach
-

4. Child → Parent Communication (@Output + EventEmitter)

Purpose

Used when the **child triggers an action** and notifies the parent.

Mechanism

- Child emits an event
- Parent listens and reacts

Example

Child Component

```
@Output() save = new EventEmitter<string>();
```

```
onSave() {  
    this.save.emit('User Saved');  
}
```

Parent Component

```
<app-editor (save)="onSaveHandler($event)"></app-editor>
```

Key Characteristics

- Event-driven
- No tight coupling
- Parent controls the logic

5. Parent Accessing Child (@ViewChild)

Purpose

Used when the parent needs to:

- Call a child method
- Access child properties

Mechanism

- Parent gets a **reference** to the child instance

Example

```
@ViewChild(ChildComponent) child!: ChildComponent;
```

```
ngAfterViewInit() {  
  this.child.resetForm();  
}
```

When to Use

- UI orchestration
- Imperative interactions

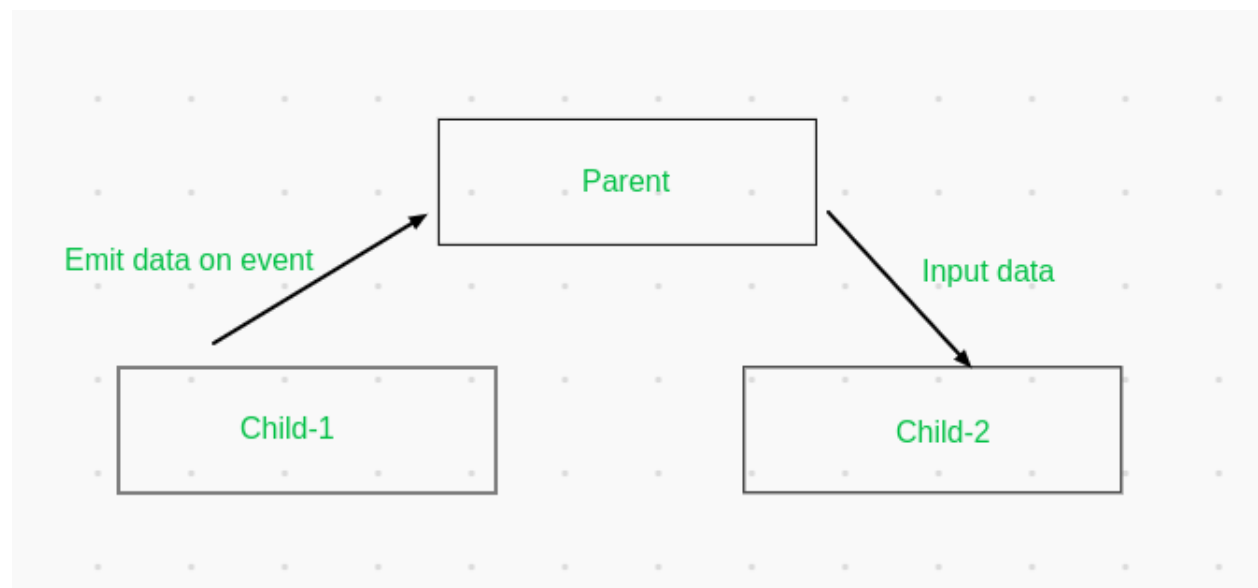
When NOT to Use

- Routine data flow
- Business logic sharing

6. Sibling → Sibling Communication (via Parent)

Recommended Pattern

Sibling components never talk directly.



Flow

Sibling A → Parent → Sibling B

Why This Matters

- Maintains unidirectional data flow
 - Avoids hidden dependencies
 - Improves testability
-

7. Unrelated Components Communication (Shared Service)

Purpose

Used when components:

- Are not related
- Exist in different modules
- Need shared state or events

Mechanism

- Singleton service
- Observable or Subject

Example Service

```
@Injectable({ providedIn: 'root' })  
export class MessageService {  
  private messageSource = new Subject<string>();  
  message$ = this.messageSource.asObservable();  
  
  send(msg: string) {  
    this.messageSource.next(msg);  
  }  
}
```

Usage in Components

```
this.messageService.send('Hello');
```

```
this.messageService.message$.subscribe(msg => {  
  console.log(msg);  
});
```

Key Characteristics

- Decoupled
 - Scalable
 - Industry-standard approach
-

8. Route-Based Communication

Use Cases

- Passing identifiers
- Navigation context
- Deep linking

Options

Method	Usage
Route Params	/users/101
Query Params	/users?id=101
Navigation State	Temporary data

Example

```
this.router.navigate(['/details', 101]);  
this.route.params.subscribe(p => console.log(p['id']));
```

Scenario	Recommended Technique
----------	-----------------------

Parent → Child	@Input
----------------	--------

Child → Parent	@Output
----------------	---------

Parent → Child (method)	@ViewChild
-------------------------	------------

Sibling	Via Parent
---------	------------

Global / Unrelated	Shared Service
--------------------	----------------

Navigation Data	Router
-----------------	--------
